

How To Write a Great Command Line Application

Dan Craig
Manager, Software Engineering
Varda Space Industries

PyCon US 2026



ME



Varda's operators reenter the capsule from Mission Control in El Segundo, California.



W-3: Making Reentry as Common as Launch



Varda Space Industries
5.5K subscribers



One too many PRs with...

```
if len(sys.argv) != 3:  
    print("This requires two input files.")  
    return
```

```
def func(..., verbose=False):  
    if verbose:  
        print("Some debugging info")
```

Why do this when you have a built-in way to make command line interfaces that

- Help the user understand how to use the program
- Provide strong validation of inputs
- Play nicely with other programs
- Are easy to test and maintain

In the beginning was the command line

How arguments reach your Python program

```
import sys  
print(sys.argv)
```

```
$ python print_args.py  
['print_args.py']
```

```
$ python print_args.py foo bar baz  
['print_args.py', 'foo', 'bar', 'baz']
```

Command Line Programs In C

```
int main(int argc, char *argv[])
```

- argc: Argument Counter
- argv: Argument Vector
- Returns: exit status integer

From man `execve`

argv is an array of pointers to strings passed to the new program as its command-line arguments. By convention, the first of these strings (i.e., argv[0]) should contain the filename associated with the file being executed.

Transforming command line input to argv

The shell transforms command line input into syscalls and argument vectors.

- Quoting
- Globbing

```
$ python print_args.py "foo bar"  
['print_args.py', 'foo bar']
```

```
$ python print_args.py books/*  
['print_args.py',  
 'books/les-miserables.txt',  
 'books/moby-dick.txt',  
 'books/war-and-peace.txt']
```

Why is “python” not in sys.argv?

```
$ python print_args.py dan  
['print_args.py', 'dan']
```

```
$ python -I print_args.py dan  
['print_args.py', 'dan']
```

Exit Status

Programs return a numeric exit status to parent process.

- Zero: success, true
- Non-zero: failure, false

```
$ python print_args.py hello world
['print_args.py', 'hello', 'world']
$ echo $?
0
```

```
$ python raises.py
Traceback (most recent call last):
  File "/home/dcraig/scratch/pycon-how-to-write-a-great-command-line-application/raises.py", line 1, in <module>
    raise Exception("womp")
Exception: womp
$ echo $?
1
```

```
$ true
$ echo $?
0
```

```
$ false
$ echo $?
1
```

equals.py – Compare two input values

```
import sys

if sys.argv[1] == sys.argv[2]:
    sys.exit(0)
sys.exit(1)
```

```
$ if python equals.py foo foo; then \
echo "they are equal" \
else echo "they are not" \
fi
they are equal
```

```
$ if python equals.py foo bar; then \
echo "they are equal" \
else
echo "they are not" \
fi
they are not
```

But it's brittle

```
$ python equals.py foo
```

```
Traceback (most recent call last):
```

```
  File "/home/dcraig/scratch/pycon-how-to-write-a-great-command-line-  
application/equals.py", line 3, in <module>
```

```
    if sys.argv[1] == sys.argv[2]:
```

```
        ~~~~~~^^^
```

```
IndexError: list index out of range
```

```
$ python equals.py foo foo bar
```

```
$ echo $?
```

```
0
```

```
$ python equals.py --help
```

```
Traceback (most recent call last):
```

```
  File "/home/dcraig/scratch/pycon-how-to-write-a-great-command-line-  
application/equals.py", line 3, in <module>
```

```
    if sys.argv[1] == sys.argv[2]:
```

```
        ~~~~~~^^^
```

```
IndexError: list index out of range
```

Better with argparse

```
import sys
from argparse import ArgumentParser

parser = ArgumentParser(
    description="Compare two values"
)
parser.add_argument("first")
parser.add_argument("second")
args = parser.parse_args()

if args.first == args.second:
    sys.exit(0)
sys.exit(1)
```

```
$ python equals.py foo
usage: equals.py [-h] first second
equals.py: error: the following arguments are
required: second
```

```
$ python equals.py foo foo bar
usage: equals.py [-h] first second
equals.py: error: unrecognized arguments: bar
```

```
$ python equals.py --help
usage: equals.py [-h] first second
```

Compare two values

positional arguments:

- first
- second

options:

- h, --help show this help message and exit

Numeric Input

```
import sys
from argparse import ArgumentParser

parser = ArgumentParser()
parser.add_argument("first")
parser.add_argument("second")
args = parser.parse_args()

if args.first > args.second:
    sys.exit(0)
sys.exit(1)
```

```
$ python greater_than.py 2 1
$ echo $?
1
```

That's not right...

Casting the input a float

```
import sys
from argparse import ArgumentParser

parser = ArgumentParser()
parser.add_argument("first", type=float)
parser.add_argument("second", type=float)
args = parser.parse_args()

if args.first > args.second:
    sys.exit(0)
sys.exit(1)
```

```
$ python greater_than.py 2 1
$ echo $?
0
```

```
$ python greater_than.py 2 foo
usage: greater_than.py [-h] first second
greater_than.py: error: argument second:
invalid float value: 'foo'
```

Much better.

```
import sys
from argparse import ArgumentParser

from dateparser import parse as parse_date

def parse_address(arg: str) -> (str, int):
    host, _, port = arg.partition(":")
    port = int(port)
    return (host, port)

parser = ArgumentParser()
parser.add_argument("--date", type=parse_date)
parser.add_argument("--address", type=parse_address)
args = parser.parse_args()

print(f>Date: {args.date}")
print(f>Address: {args.address}")
```

```
$ python custom_types.py --date "last month" --address 127.0.0.1:8080
Date: 2026-04-10 19:33:21.975881
Address: ('127.0.0.1', 8080)
```

Testability

- Need testable units
- Should not run when imported
- Can't call `sys.exit()` in the middle of a test

```
import sys
from argparse import ArgumentParser

parser = ArgumentParser()
parser.add_argument("first", type=float)
parser.add_argument("second", type=float)
args = parser.parse_args()

if args.first > args.second:
    sys.exit(0)
sys.exit(1)
```

Adding testability with a main() function

```
import sys
from argparse import ArgumentParser

def main(raw_args: list[str]) -> int:
    parser = ArgumentParser()
    parser.add_argument("first")
    parser.add_argument("second")
    args = parser.parse_args(raw_args)

    if args.first > args.second:
        return 0
    return 1

if __name__ == "__main__":
    sys.exit(main(sys.argv[1:]))
```

```
from greater_than import main

def test_greater_than():
    assert main(["2", "1"]) == 0

def test_not_greater_than():
    assert main(["2", "3"]) == 1
```

- Importable function with testable return value
- Script execution isolated with `__main__`
- No need to subprocess

File Inputs and Printing Output

wc - print newline, word, and byte counts for each file

```
$ wc books/*  
 73590  568652 3369210 books/les-miserables.txt  
 22314  215845 1276266 books/moby-dick.txt  
 66041  566333 3359613 books/war-and-peace.txt  
161945 1350830 8005089 total
```

```
$ wc -l -w books/*  
 73590  568652 books/les-miserables.txt  
 22314  215845 books/moby-dick.txt  
 66041  566333 books/war-and-peace.txt  
161945 1350830 total
```

```
import sys
from argparse import ArgumentParser
from pathlib import Path

def main(raw_args: list[str]) -> int:
    parser = ArgumentParser(description="Count newlines, words, and characters")
    parser.add_argument("filename", type=Path)
    args = parser.parse_args(raw_args)

    text = args.filename.read_text()
    line_count = len(text.splitlines())
    word_count = len(text.split())
    char_count = len(text)

    print(f"{line_count} {word_count} {char_count} {str(args.filename)}")
    return 0

if __name__ == "__main__":
    sys.exit(main(sys.argv[1:]))
```

```
import sys
from argparse import ArgumentParser, FileType

def main(raw_args: list[str]) -> int:
    parser = ArgumentParser(description="Count newlines, words, and characters")
    parser.add_argument("infile", type=FileType('r'), default=sys.stdin, nargs="?")
    args = parser.parse_args(raw_args)

    line_count = word_count = char_count = 0
    for line in args.infile:
        line_count += 1
        word_count += len(line.split())
        char_count += len(line)

    print(f"{line_count} {word_count} {char_count} {str(args.infile.name)}")
    return 0

if __name__ == "__main__":
    sys.exit(main(sys.argv[1:]))
```

```
import sys
from argparse import ArgumentParser, FileType

def main(raw_args: list[str]) -> int:
    parser = ArgumentParser(description="Newline, word, and other counts for a file")
    parser.add_argument("infile", type=FileType('r'), default=[sys.stdin], nargs="*")
    args = parser.parse_args(raw_args)

    for infile in args.infiles:
        line_count = word_count = char_count = 0
        for line in args.infile:
            line_count += 1
            word_count += len(line.split())
            char_count += len(line)

        print(f"{line_count} {word_count} {char_count} {str(args.infile.name)}")
    return 0

if __name__ == "__main__":
    sys.exit(main(sys.argv[1:]))
```



argparse.FileType opens a file and never closes it #58032

#124664



serhiy-storchaka on Jan 1, 2024

Member ...

The current design has too many flaws. Open files are leaked if `argparse` itself fails or the user code fails or quits before it has chance to close all files. Output files are always rewritten in case of error. Input files can remain open for too long time and consume the limited resource of file descriptors. And solving it in the user code is difficult even if we ignore errors that can occur during parsing (see for example the pickletools CLI [#85567](#)).

All solutions (making file opening lazy (and maybe explicit) or making the parser or the namespace object a context manager) require changing the user code. Minimizing changes in the user code increases chance of writing user code that does not do all thing properly.

On other hand, explicit opening and closing files is not much more complex that proper use of `FileType` or its replacement. The only non-trivial thing that `FileType` does in comparison with the plain `open()` is returning the standard stream (depending on mode) if the argument is `"-"`. We can provide an utility function for this, although the code is only non-trivial because it is general, for any concrete mode string it is simpler.

I propose to deprecate `argparse.FileType`. Its existence in the `stdlib` implies that it is a preferable way to handle file arguments, but it is not so. You have less issues if you do not use it.



gvanrossum on Jan 1, 2024

Member ...

+1. Well argued. I'm not even sure we need a helper for `'-'`; that API would be a bit convoluted, since closing `stdin` is fraught.



FileType objects

```
class argparse.FileType(mode='r', bufsize=-1, encoding=None, errors=None)
```

The `FileType` factory creates objects that can be passed to the `type` argument of [ArgumentParser.add_argument\(\)](#). Arguments that have `FileType` objects as their type will open command-line arguments as files with the requested modes, buffer sizes, encodings and error handling (see the [open\(\)](#) function for more details):

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--raw', type=argparse.FileType('wb', 0))
>>> parser.add_argument('out', type=argparse.FileType('w', encoding='UTF-8'))
>>> parser.parse_args(['--raw', 'raw.dat', 'file.txt'])
Namespace(out=<_io.TextIOWrapper name='file.txt' mode='w' encoding='UTF-8'>, raw=<_io.File
```

`FileType` objects understand the pseudo-argument `'-'` and automatically convert this into [sys.stdin](#) for readable `FileType` objects and [sys.stdout](#) for writable `FileType` objects:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('infile', type=argparse.FileType('r'))
>>> parser.parse_args(['-'])
Namespace(infile=<_io.TextIOWrapper name='<stdin>' encoding='UTF-8'>)
```

Note: If one argument uses `FileType` and then a subsequent argument fails, an error is reported but the file is not automatically closed. This can also clobber the output files. In this case, it would be better to wait until after the parser has run and then use the [with](#)-statement to manage the files.

Changed in version 3.4: Added the `encoding` and `errors` parameters.

Deprecated since version 3.14.

Alternatives to FileType?

```
def PathOrStreamType(stream):
    """Factory function to create a type that will
    use a stream if the argument is "-", otherwise
    cast the argument into a Path."""
    def _path_or_stream(arg):
        if arg == "-":
            return stream
        return Path(arg)

def open_file_or_io(file_path_or_io, *args, **kwargs):
    """Return the open() context manager for a file,
    or a null context manager an existing file object."""
    if isinstance(file_path_or_io, IOBase):
        return nullcontext(file_path_or_io)
    return open(file_path_or_io, *args, **kwargs)
```

```
parser = ArgumentParser()
parser.add_argument(
    "infile",
    type=PathOrStreamType(sys.stdin),
    default=sys.stdin,
    nargs="?"
)

args = parser.parse_args([])
args = parser.parse_args(["-"])
args = parser.parse_args(["foo.txt"])

with open_file_or_io(args.infile, 'r') as fobj:
    for line in fobj:
        ...
```

Optional arguments

```
def main(raw_args: list[str]) -> int:
    parser = ArgumentParser(description="Count newlines, words, and characters")
    parser.add_argument("infile", type=FileType('r'), default=[sys.stdin], nargs="*")
    parser.add_argument("-l", "--lines", action="store_true", help="Print the newline count.")
    parser.add_argument("-w", "--words", action="store_true", help="print the word counts")
    args = parser.parse_args(raw_args)

    for infile in args.infiles:
        line_count = word_count = char_count = 0
        for line in args.infile:
            line_count += 1
            word_count += len(line.split())
            char_count += len(line)

        no_options = not any([args.lines, args.words])
        outputs = []
        if no_options or args.lines:
            outputs.append(line_count)
        if no_options or args.words:
            outputs.append(word_count)
        if no_options:
            outputs.append(char_count)
        if args.infile != sys.stdin:
            outputs.append(args.infile.name)
        print(f"{' '.join(map(str, outputs))}")
```

```
def main(raw_args: list[str]) -> int:
    parser = ArgumentParser(description="Count newlines, words, and characters")
    parser.add_argument("infile", type=FileType('r'), default=[sys.stdin], nargs="*")
    parser.add_argument("-l", "--lines", action="store_true", help="Print the newline count.")
    parser.add_argument("-w", "--words", action="store_true", help="print the word counts")
    args = parser.parse_args(raw_args)
```

```
for infile in args.infiles:
    line_count = word_count = char_count = 0
    for line in args.infile:
        line_count += 1
        word_count += len(line.split())
        char_count += len(line)
```

```
no_options = not any([args.lines, args.words])
outputs = []
if no_options or args.lines:
    outputs.append(line_count)
if no_options or args.words:
    outputs.append(word_count)
if no_options:
    outputs.append(char_count)
if args.infile != sys.stdin:
    outputs.append(args.infile.name)
print(f"{' '.join(map(str, outputs))}")
```

Business Logic

Output Formatting

Input Collection

```
def counts(infile):  
    line_count = word_count = char_count = 0  
    for line in infile:  
        line_count += 1  
        word_count += len(line.split())  
        char_count += len(line)  
    return line_count, word_count, char_count
```

```
def main(raw_args: list[str]) -> int:  
    parser = ArgumentParser(description="Count newlines, words, and characters")  
    ...  
    args = parser.parse_args(raw_args)  
  
    for infile in args.infiles:  
        line_count, word_count, char_count = counts(args.infile)  
  
        no_options = not any([args.lines, args.words])  
        outputs = []  
        if no_options or args.lines:  
            outputs.append(line_count)  
        if no_options or args.words:  
            outputs.append(word_count)  
        if no_options:  
            outputs.append(char_count)  
        if args.infile != sys.stdin:  
            outputs.append(args.infile.name)  
        print(f"{' '.join(map(str, outputs))}")
```

```
def counts(infile):  
    line_count = word_count = char_count = 0  
    for line in infile:  
        ...  
    return line_count, word_count, char_count
```

```
def do(infiles, lines, words):  
    for infile in infiles:  
        line_count, word_count, char_count = counts(infile)  
  
        no_options = not any([lines, words])  
        outputs = []  
        if no_options or lines:  
            outputs.append(line_count)  
        if no_options or words:  
            outputs.append(word_count)  
        if no_options:  
            outputs.append(char_count)  
        if infile != sys.stdin:  
            outputs.append(infile.name)  
        yield outputs
```

```
def main(raw_args: list[str]) -> int:  
    ...  
    for output in do(args.infiles, args.lines, args.words):  
        print(f"{' '.join(map(str, output))}")
```

```

def counts(infile):
    line_count = word_count = char_count = 0
    for line in infile:
        ...
    return line_count, word_count, char_count

def do(*, infiles, lines, words):
    for infile in infiles:
        line_count, word_count, char_count = counts(infile)

        no_options = not any([lines, words])
        outputs = []
        if no_options or lines:
            outputs.append(line_count)
        if no_options or words:
            outputs.append(word_count)
        if no_options:
            outputs.append(char_count)
        if infile != sys.stdin:
            outputs.append(infile.name)
        yield outputs

def main(raw_args: list[str]) -> int:
    ...
    for output in do(infiles=args.infiles, lines=args.lines, words=args.words):
        print(f"{' '.join(map(str, output))}")

```

```
def do(*, infiles, lines, words):  
    for infile in infiles:  
        line_count, word_count, char_count = counts(infile) } Business Logic  
  
        no_options = not any([lines, words])  
        outputs = []  
        if no_options or lines:  
            outputs.append(line_count)  
        if no_options or words:  
            outputs.append(word_count)  
        if no_options:  
            outputs.append(char_count)  
        if infile != sys.stdin:  
            outputs.append(infile.name)  
        yield outputs
```

Output Formatting

```
def main(raw_args: list[str]) -> int:  
    ...  
    args = parser.parse_args(raw_args) } Input Collection
```

```
for output in do(**vars(args)):  
    print(f"{' '.join(map(str, output))}") } Writing Output
```

```
def main(raw_args: list[str]) -> int:
    """
    Count newlines, words, and characters
    """
    parser = ArgumentParser(description=main.__doc__)
    parser.add_argument("infile", type=FileType('r'), default=[sys.stdin], nargs="*")
    parser.add_argument("-l", "--lines", action="store_true", help="Print the newline count.")
    parser.add_argument("-w", "--words", action="store_true", help="print the word counts")
    args = parser.parse_args(raw_args)

    for output in do(**vars(args)):
        print(f"{' '.join(map(str, output))}")

    return 0

if __name__ == "__main__":
    sys.exit(main(sys.argv[1:]))
```

Playing nicely with other programs

```
import sys
from argparse import ArgumentParser
from time import sleep

def fibonnaci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

def main(raw_args: list[str]) -> int:
    """Prints the fibonnaci sequence"""
    parser = ArgumentParser(description=main.__doc__)
    parser.parse_args(raw_args)

    for value in fibonnaci():
        print(value)
        sleep(0.2)

    return 0

if __name__ == "__main__":
    sys.exit(main(sys.argv[1:]))
```

```
$ python fibonacci.py
```

```
0
```

```
1
```

```
1
```

```
2
```

```
3
```

```
^CTraceback (most recent call last):
```

```
  File "/home/dcraig/scratch/pycon-how-to-write-a-great-command-line-  
application/fibonacci.py", line 33, in <module>
```

```
    sys.exit(main(sys.argv[1:]))
```

```
~~~~^^^^^^^^^^^^^^^^
```

```
  File "/home/dcraig/scratch/pycon-how-to-write-a-great-command-line-  
application/fibonacci.py", line 28, in main
```

```
    sleep(0.2)
```

```
~~~~~^^^^
```

```
KeyboardInterrupt
```

```
if __name__ == "__main__":  
    try:  
        sys.exit(main(sys.argv[1:]))  
    except KeyboardInterrupt:  
        sys.exit(1)
```

```
$ python fibonacci.py
```

```
0
```

```
1
```

```
1
```

```
2
```

```
3
```

```
^C
```

```
$ echo $?
```

```
1
```

```
$ python fibonacci.py | head -n 10  
^CException ignored on flushing sys.stdout:  
BrokenPipeError: [Errno 32] Broken pipe
```

```
def main(raw_args: list[str]) -> int:
    """Prints the fibonnaci sequence"""
    parser = ArgumentParser(
        description=main.__doc__
    )
    parser.parse_args(raw_args)

    for value in fibonacci():
        print(value, flush=True)
        sleep(0.2)

    return 0
```

```
$ python fibonacci.py | head -n 5
```

```
0
```

```
1
```

```
1
```

```
2
```

```
3
```

```
Traceback (most recent call last):
```

```
  File "/home/dcraig/scratch/pycon-how-to-
write-a-great-command-line-
application/fibonacci.py", line 34, in <module>
    sys.exit(main(sys.argv[1:]))
```

```
~~~~^^^^^^^^^^^^^^^^^^
```

```
  File "/home/dcraig/scratch/pycon-how-to-
write-a-great-command-line-
application/fibonacci.py", line 27, in main
    print(value, flush=True)
```

```
~~~~^^^^^^^^^^^^^^^^^^
```

```
BrokenPipeError: [Errno 32] Broken pipe
Exception ignored on flushing sys.stdout:
BrokenPipeError: [Errno 32] Broken pipe
```

```
if __name__ == "__main__":  
    try:  
        sys.exit(main(sys.argv[1:]))  
    except KeyboardInterrupt:  
        sys.exit(1)  
    except BrokenPipeError:  
        devnull = os.open(os.devnull, os.O_WRONLY)  
        os.dup2(devnull, sys.stdout.fileno())  
        sys.exit(0)
```

```
$ python fibonacci.py | head -n 10
```

```
0  
1  
1  
2  
3  
5  
8  
13  
21  
34
```

Logging and Verbosity

How many of you have seen something like this?

```
def func(..., verbose=False):  
    ...  
    if verbose:  
        print("Some debugging info")
```

Slightly better

```
print("Some debugging info", file=sys.stderr)
```

Leveraging Logging

```
import logging

logger = logging.getLogger(__name__)

def do(*, infiles, upper: bool):
    logger.info("Helpful details")

def main(raw_args: list[str]) -> int:
    """Repeats a line, optionally uppercasing it."""
    parser = ArgumentParser(description=main.__doc__)
    parser.add_argument('-v', '--verbosity', action="count", default=0)
    args = vars(parser.parse_args(raw_args))

    verbosity = args.pop('verbosity')
    level = max(logging.ERROR - verbosity * 10, logging.NOTSET)
    logging.basicConfig(level=level, format="%(levelname)s %(message)s")

    for output in do(**args):
        print(output, end='', flush=True)
```

Third Party Libraries

Click

- Leverages decorators on functions to reduce the boilerplate.
- Has great tools for testing.

```
import sys
import click

@click.command()
@click.argument("first", type=str)
@click.argument("second", type=str)
def main(first: str, second: str) -> int:
    return 0 if first > second else 1

if __name__ == "__main__":
    sys.exit(main())
```

tyro

- Leverages dataclasses, type annotations, docstrings, and function argument defaults to reduce the boilerplate.

```
import sys
import tyro
from dataclasses import dataclass

@dataclass
class Args:
    first: str
    second: str

def main(args: Args) -> int:
    return 0 if args.first > args.second else 1

if __name__ == "__main__":
    parsed = tyro.cli(Args)
    sys.exit(main(parsed))
```

Typer

- From the makers of FastAPI, follows similar principles
- Leaning into type annotations
- Built on top of Click.

```
import sys
import typer

app = typer.Typer(add_completion=False)

@app.command()
def main(first: str, second: str) -> int:
    return 0 if first > second else 1

if __name__ == "__main__":
    sys.exit(app())
```

Running your script like a program

Executing a script with a shebang

Make the script executable and run it

```
$ chmod +x script.py  
$ ./script.py
```

Hard-coded to only one absolute path

```
#!/usr/bin/python
```



Find a python executable on your PATH, including activated virtual envs

```
#!/usr/bin/env python
```



Let uv select the python executable and manage your dependencies automatically.

```
#!/usr/bin/env -S uv run --script
```



Leveraging uv

Manage Python version and dependencies using PEP 723 Inline script metadata

```
#!/usr/bin/env -S uv run --script
#
# /// script
# requires-python = ">=3.13"
# dependencies = ["requests"]
# ///
import logging
import os
import sys
from argparse import ArgumentParser

...
```

Good argparse patterns
in summary

```

#! /usr/bin/env -S uv run --script
#
# /// script
# requires-python = ">=3.13"
# dependencies = ["requests"]
# ///
import logging
import os
import sys
from argparse import ArgumentParser, FileType

logger = logging.getLogger(__name__)

def do(*, infiles, upper: bool):
    for infile in infiles:
        for line in infile:
            if upper:
                yield line.upper()
            else:
                yield line

def main(raw_args: list[str]) -> int:
    """Repeats a line"""

```

Use a shebang with uv to specify your third party dependencies.

Use a do() function that takes all your CLI's arguments as keyword-only arguments and yields output a line a time.

```
def main(raw_args: list[str]) -> int:
```

Use a main function that takes a list of args and returns the status code

```
    """Repeats a line"""
```

```
    parser = ArgumentParser(description=main.__doc__)
```

Reuse your docstring for your description

```
    parser.add_argument(
```

```
        'infile',
```

```
        type=FileType('r'),
```

```
        default=[sys.stdin],
```

```
        nargs="*")
```

```
    parser.add_argument('-v', '--verbosity', action="count", default=0)
```

```
    parser.add_argument('-u', '--upper', action="store_true")
```

```
    args = vars(parser.parse_args(raw_args))
```

```
    verbosity = args.pop('verbosity')
```

```
    level = max(logging.ERROR - verbosity * 10, logging.NOTSET)
```

```
    logging.basicConfig(level=level, format="%(levelname)s %(message)s")
```

Use logging for verbosity

```
    for output in do(**args):
```

```
        print(output, end='', flush=True)
```

```
    return 0
```

Use keyword expansion of args of the do function.

```
for output in do(**args):  
    print(output, end='', flush=True)
```

```
return 0
```

```
if __name__ == "__main__":
```

```
    try:
```

```
        sys.exit(main(sys.argv[1:]))
```

```
    except KeyboardInterrupt:
```

```
        sys.exit(1)
```

```
    except BrokenPipeError:
```

```
        devnull = os.open(os.devnull, os.O_WRONLY)
```

```
        os.dup2(devnull, sys.stdout.fileno())
```

```
        sys.exit(0)
```

} Call `sys.exit()` with the `main()` function's exit status return value

} Handle `KeyboardInterrupt` and `BrokenPipeError` to make your program play nicely with others

Good argparse patterns

- Use a shebang with uv to specify your third party dependencies.
- Use a main function that takes a list of raw arguments and returns the exit code.
- Use `main.__doc__` to deduplicate your docstrings and your help text.
- Call your main function under dunder main and pass its return value to `sys.exit()`.
- Catch and handle `KeyboardInterrupt` and `BrokenPipeErrors` to make your program play nicely with others.
- Use a `do()` function that takes all of your argparse arguments using keyword-only arguments with `**vars()`, and yield each line or chunk of output.
- Use logging for verbose debugging output.



<https://github.com/drcraig/how-to-write-great-command-line-application-pycon-2026>

Link to Varda @ Pycon In-Space
Happy Hour, tonight, 7:30pm
on the README